

Tic Tac Toe (tkinter GUI) — Deep Dive

Project explainer document

Contents

1	What this document is	1
2	Project purpose and history	1
3	Architecture overview	1
4	The board representation	2
4.1	Win lines	2
5	Core game functions (game.py)	2
6	The AI opponent: minimax	3
6.1	What “unbeatable” means here, precisely	3
6.2	Beginner-safe explanation of minimax	3
6.3	A worked mini-example	3
6.4	Why depth is subtracted or added to the score	4
6.5	The actual recursive function	4
6.6	Alpha-beta pruning	5
6.7	best_ai_move: the function the GUI actually calls	5
7	The GUI (main.py)	6
7.1	Mode toggle	6
7.2	Turn flow in Vs AI mode	6
7.3	Score tracking, extended	7
8	Testing and verification	7
8.1	Unit tests (tests/test_ai.py)	7
8.2	Exhaustive and simulated unbeatability proof (tests/verify_unbeatable.py)	7
8.3	GUI smoke test	8
9	Summary of what changed from the previous version	8

1 What this document is

This is a working reference for understanding every part of the Tic Tac Toe project, written so that someone with no prior exposure to the codebase (or even to some of the underlying concepts) can follow it end to end. It covers the original game and GUI, and the newest addition: a single-player mode against an unbeatable computer opponent built with the **minimax algorithm**.

2 Project purpose and history

The project started as a console-only practice script (“Day 83” of a coding exercise series) that printed a numbered board and read moves from `input()`. That version had a real bug: its win check only ever compared cells against the mark `X`, so a completed line for `O` was never detected and the game kept running as if nothing had happened.

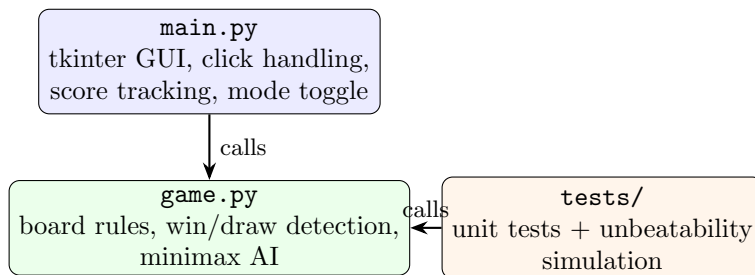
The rewrite kept the original goal (represent a 3x3 board and detect a win across all eight possible lines without a third-party library) but fixed the bug and replaced the terminal loop with a real tkinter graphical window. This document covers that GUI version plus the AI opponent added afterward.

3 Architecture overview

The project is split into three files, each with one job:

- `game.py` — all game rules and the AI’s decision-making. No tkinter import anywhere in this file, no `print` or `input` calls. Pure functions that take a board (a plain Python list) and return an answer.
- `main.py` — the tkinter GUI. Builds the window, the 3x3 grid of buttons, the mode toggle, the status and score labels, and wires user clicks (and the AI’s automatic move) to the functions in `game.py`.
- `tests/` — automated checks: unit tests for individual AI decisions (`test_ai.py`) and a standalone script that plays out hundreds of full games to confirm the AI never loses (`verify_unbeatable.py`).

Why split the logic out of the GUI at all? A GUI framework like tkinter is hard to test automatically because it wants to draw actual windows and respond to real mouse/keyboard events. By keeping every rule of the game (whose turn it is, who won, what the AI should do next) in plain functions that only touch a Python list, those rules can be tested directly and quickly, with no window ever opening. `main.py` then becomes a thin layer that only answers “how do I draw this” and “what button did the user click,” and always defers to `game.py` for “what does this move mean.”



4 The board representation

The board is a flat Python list of 9 strings, indexed 0 through 8:

0	1	2
3	4	5
6	7	8

Each cell holds `"X"`, `"O"`, or `" "` (empty, the constant `EMPTY`). Using a flat list instead of a 2D grid keeps every function simple: no row/column math is needed except when defining which triples of indices count as a win.

4.1 Win lines

```
WIN_LINES = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # columns
    (0, 4, 8), (2, 4, 6), # diagonals
]
```

`check_winner` loops over these eight triples and checks whether all three cells in a triple are non-empty and equal. This single loop replaces what the original console version did as eight separate, hardcoded or-chained comparisons (and only against X, which was the bug). Looping over a list of index triples handles both X and O with the same code path, so the class of bug (“only checks for one player”) cannot recur silently.

5 Core game functions (`game.py`)

- `new_board()` — returns a fresh list of 9 empty strings.
- `make_move(board, index, mark)` — places a mark if the cell is empty and the index is in range 0–8; returns `True/False` so the caller knows whether the move was legal.
- `check_winner(board)` — returns (`winner mark, line`) or `None`.
- `is_draw(board)` — true when the board is full and `check_winner` found nothing.
- `is_game_over(board)` — true if either of the above is true.
- `available_moves(board)` — the list of empty cell indices, in board order. Used by the AI to know which moves it’s allowed to try.
- `other_mark(mark)` — flips “X” to “O” and vice versa. A tiny helper, but it removes a whole class of copy-paste bugs where the “whose turn is next” logic gets typed out by hand in multiple places and drifts out of sync.

6 The AI opponent: minimax

6.1 What “unbeatable” means here, precisely

An unbeatable Tic Tac Toe player is one that, playing from any legal position, never loses: it either wins (if the position allows a forced win) or draws (if the position is a proven draw with correct play, which the empty board is). This is a much stronger and more precisely defined claim than “plays pretty well,” and it is exactly what minimax guarantees, for a full-information game with no hidden state and no randomness, like Tic Tac Toe.

6.2 Beginner-safe explanation of minimax

Imagine every possible way the rest of the game could unfold from the current board as a tree: the current board is the root, each of its children is the board after one more move, and so on down to boards where the game is over (someone won, or it’s a draw). This is called the **game tree**.

Game tree: a diagram of every possible sequence of future moves from a given position, branching at each turn into one child per legal move.

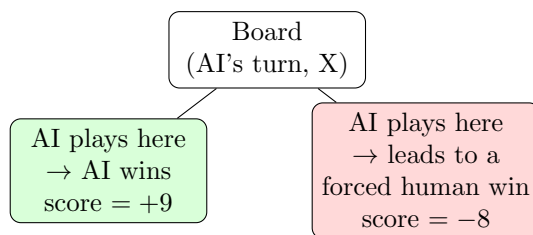
Minimax explores this entire tree and assigns a score to every finished (“terminal”) board: a win for the AI scores positively, a win for the human scores negatively, and a draw scores zero. It then works backward up the tree:

- On the AI’s turns, it assumes the AI will pick whichever child has the *highest* score (the AI “maximizes”).
- On the human’s turns, it assumes the human will pick whichever child has the *lowest* score, i.e. the worst outcome for the AI (the AI *plans for the human to play their best move against it*, not a lucky or careless one – this is what “minimize” means, and it’s the core trick that makes the AI safe against a strong opponent, not just a weak one).

The name “minimax” is literally this: the AI *maximizes* its own score, assuming the opponent will *minimize* it. Because every branch is visited before a decision is made, and because Tic Tac Toe is small enough to enumerate completely (at most $9! = 362,880$ move orders, and far fewer once finished games stop branching), this is not a heuristic guess – it is a complete, correct answer to “what is the best possible move here.”

6.3 A worked mini-example

Consider a tiny slice of the tree where it is the AI’s (X’s) turn and there are only two legal moves left, each leading to a different kind of ending:



Minimax computes the score under *both* branches (recursing further down each one until the game ends), then the AI, maximizing, picks the left branch because $+9 > -8$. In a real game there are up to 9 branches at the first move and the tree is many levels deep, but the principle is identical at every level: compute both children fully, then pick the best one for whoever’s turn it is.

6.4 Why depth is subtracted or added to the score

```
def _score_terminal(board, ai_mark, depth):
    result = check_winner(board)
    if result is None:
        return 0
    winner = result[0]
    return (10 - depth) if winner == ai_mark else (depth - 10)
```

A plain win/loss/draw score of $+1/-1/0$ is already enough to make the AI unbeatable. But it would be indifferent between winning in 2 moves and winning in 8, or between losing (if that were ever forced) immediately versus after a long fight. Subtracting the search depth from a win score, and adding it to a loss score, breaks that tie: the AI prefers the *fastest* win and the *slowest* loss among otherwise equally-good options. This never changes whether the AI can be beaten (that answer only depends on the sign of the score); it just makes an already-unbeatable AI play with better “style,” finishing you off as soon as a forced win is available instead of stalling.

6.5 The actual recursive function

```
def minimax(board, current_mark, ai_mark, depth=0, alpha=-math.inf, beta=math.inf):
    if is_game_over(board):
        return _score_terminal(board, ai_mark, depth), None

    moves = available_moves(board)
    best_move = moves[0]

    if current_mark == ai_mark:
        best_score = -math.inf
        for move in moves:
            board[move] = current_mark
            score, _ = minimax(board, other_mark(current_mark), ai_mark, depth + 1, alpha,
                               beta)
            board[move] = EMPTY
            if score > best_score:
                best_score, best_move = score, move
            alpha = max(alpha, best_score)
            if beta <= alpha:
                break
    else:
        best_score = math.inf
        for move in moves:
            board[move] = current_mark
            score, _ = minimax(board, other_mark(current_mark), ai_mark, depth + 1, alpha,
                               beta)
            board[move] = EMPTY
            if score < best_score:
                best_score, best_move = score, move
            beta = min(beta, best_score)
            if beta <= alpha:
                break

    return best_score, best_move
```

Walking through it:

1. **Base case:** if the board is already finished, stop recursing and return its terminal score. This is what eventually stops the recursion at every leaf of the game tree.
2. **Try every legal move:** for the mark whose turn it is, place that mark in each empty cell in turn (`board[move] = current_mark`), recurse to see how the rest of the game would play out from there, then *undo* the move (`board[move] = EMPTY`) before trying the next one. This “try it, recurse, undo it” pattern is called **backtracking**: it explores the tree using a single mutable board instead of copying the board at every node, which is both simpler and much faster.
3. **Maximize or minimize:** on the AI’s simulated turn, keep the move with the highest resulting score; on the opponent’s simulated turn, keep the move with the lowest resulting score.

Backtracking: exploring a decision tree by making a tentative choice, recursing into what happens next, and then reversing that choice before trying the next alternative – so the same data structure (here, one board list) can represent every branch of the tree in turn without needing a copy per branch.

6.6 Alpha-beta pruning

The **alpha** and **beta** parameters implement **alpha-beta pruning**, an optimization that does not change the answer minimax gives, only how much of the tree it has to look at to get there.

Alpha-beta pruning: while searching, keep track of the best score the maximizing side can already guarantee (**alpha**) and the best score the minimizing side can already guarantee (**beta**). If, partway through exploring a branch, it becomes clear that branch cannot possibly beat what the other side can already guarantee elsewhere, stop exploring it immediately – the rest of that branch’s detail cannot change the final decision.

Concretely: if the AI is maximizing and has already found a move worth **alpha**, and while checking the human’s best reply to some other move it discovers the human can hold the AI to **beta** $\leq$ **alpha**, there is no point computing the human’s remaining replies in that branch – the human already has a reply good enough to make the AI prefer a different move. The `if beta <= alpha: break` lines are exactly this cutoff.

On a board as small as Tic Tac Toe’s, plain minimax (no pruning at all) is already fast enough to be imperceptible from the very first move (measured at under 0.1 seconds on the empty board, the single most expensive position to search, since it has the most remaining moves). Alpha-beta pruning is included anyway because it is the standard, correct way to speed up minimax and it costs nothing in code complexity, but note it is a performance optimization, not a correctness requirement, for a game this small.

6.7 best_ai_move: the function the GUI actually calls

```
def best_ai_move(board, ai_mark):
    if is_game_over(board):
        return None
    _, move = minimax(board, current_mark=ai_mark, ai_mark=ai_mark)
    return move
```

This is a thin, GUI-friendly wrapper: give it the current board and which mark the AI is playing, and it returns the single best cell index to play, or **None** if the game has already ended. The GUI never has to know anything about scores, depth, or recursion – it just gets a move.

7 The GUI (main.py)

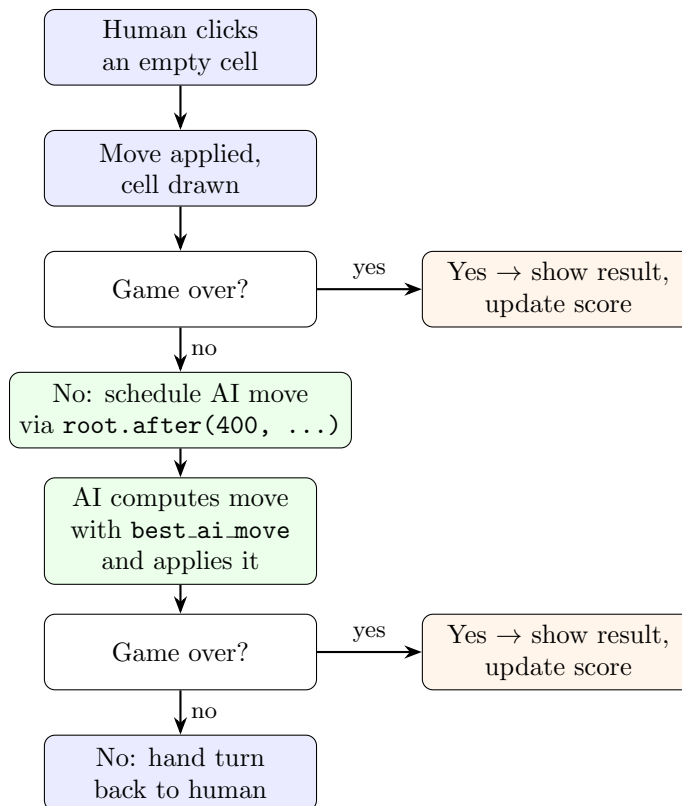
7.1 Mode toggle

The window now has two radio-button rows above the board:

- **Two player / Vs AI** – selects `self.mode`, a tkinter `StringVar`.
- When “Vs AI” is selected, a second row appears (**You play: X (first) / O (second)**), letting the human choose either mark. This row is hidden (`pack_forget()`) in two-player mode and re-shown (`pack(..., before=...)`) in Vs AI mode, keeping the same visual slot in the layout either way.

Changing either radio button calls `_on_mode_change`, which updates the visible score line and resets the board for the new mode.

7.2 Turn flow in Vs AI mode



The 400ms delay before the AI's move (`AI_MOVE_DELAY_MS` in `main.py`) is a deliberate UX choice: the move itself is computed instantly (minimax on an empty 3x3 board finishes in well under a tenth of a second), so without a delay the AI's reply would feel like an instantaneous, robotic snap. Scheduling it with `root.after` instead of a blocking `time.sleep` keeps the tkinter event loop responsive during the wait (the window can still be redrawn, moved, etc.) and a status label reading "AI is thinking..." is shown for that window, and a `self.ai_thinking` flag rejects any stray human clicks that land during it.

If the human chooses to play 0, the AI (playing X, which always moves first in this project's convention) makes its opening move automatically as soon as the board resets, using the same scheduled-delay mechanism.

7.3 Score tracking, extended

The original single score dictionary (X wins / 0 wins / draws) is still used for two-player mode. A second dictionary, `scores_vs_ai = {"human": 0, "ai": 0, "draws": 0}`, tracks results against the AI, independent of which mark the human happened to be playing in a given round. The score label switches which dictionary it reads based on the current mode.

8 Testing and verification

8.1 Unit tests (tests/test_ai.py)

Six focused tests call `best_ai_move` and `minimax` directly on hand-built boards, with no tkinter involved at all:

- Taking an immediate winning move when one is available.

- Blocking the human’s immediate winning move when the AI has no win of its own available yet.
- Preferring its own immediate win over blocking the human’s, when both are available in the same position (a case where a naive “always block first” AI would get it wrong).
- Confirming the empty board evaluates to a score of exactly 0 (a theoretical draw with perfect play from both sides) and that the chosen opening is a corner or the center, the two game-theoretically sound openings.
- Refusing to return a move on a full board (returns `None`).
- Picking the center as the correct reply to a lone opening corner move.

All six pass.

8.2 Exhaustive and simulated unbeatability proof (`tests/verify_unbeatable.py`)

Unit tests on a handful of hand-picked boards prove the AI is *correct* on those boards, but they cannot by themselves prove the AI *never* loses across the whole game. `verify_unbeatable.py` closes that gap with three separate, much stronger checks, run for both possible starting orders (human moves first, AI moves first):

1. **Exhaustive search over every human strategy.** Rather than sampling, this recursively branches into *every* legal move the human could make at *every* one of its turns, with the AI always replying via `best_ai_move`. This visits every distinct game reachable by any sequence of human choices – not a sample, a complete enumeration.
2. **1000 games per starting order with the human playing uniformly random legal moves.**
3. **Both sides playing optimally** (the “human” side also uses `minimax`), which game theory says must always end in a draw.

Actual results from running this script (reproducible with python3 `tests/verify_unbeatable.py`):

- Exhaustive search, human moves first: 569 distinct games explored – AI won 386, drew 183, **lost 0**.
- Exhaustive search, AI moves first: 73 distinct games explored – AI won 71, drew 2, **lost 0**.
- 1000 random-play games, human moves first: AI won 789, drew 211, **lost 0**.
- 1000 random-play games, AI moves first: AI won 996, drew 4, **lost 0**.
- Optimal vs. optimal, human moves first: draw (as game theory predicts).
- Optimal vs. optimal, AI moves first: draw (as game theory predicts).

Across all of these, the AI lost **zero** games. Combined with the exhaustive check (which is not a sample, but a complete traversal of every human move sequence for each starting order), this is a direct, reproducible demonstration of unbeatability, not an inference from “it seemed to play well.”

8.3 GUI smoke test

A short script drives `TicTacToeApp` programmatically (without a human clicking): it switches to Vs AI mode with the human playing X, simulates a human move, waits past the AI’s scheduled delay, and checks the AI replied; then switches the human to O and confirms the AI auto-moves first; then switches back to two-player mode and confirms it still behaves independently. No exceptions were raised in any step.

9 Summary of what changed from the previous version

- Added `available_moves`, `other_mark`, `_score_terminal`, `minimax`, and `best_ai_move` to `game.py`. No existing function's behavior changed.
- Added a mode toggle, mark-choice controls, an automatic delayed AI move, and a second score dictionary to `main.py`. Two-player mode is unchanged in behavior.
- Added `tests/test_ai.py` and `tests/verify_unbeatable.py`.